Minimum Spanning Trees, Prim's and Dijkstra's Algorithms

Greedy Strategies, Weighted Graphs, Shortest Paths and C Implementations

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

Recap: Graph Algorithms

Previously, we studied:

- Disjoint-set representation.
- MAKE-SET, FIND and UNION.
- Path compression.
- Union by rank and union by size.
- Minimum spanning trees.
- Kruskal's greedy algorithm.

Today's Focus

How can we construct an MST by growing a single tree?
How can we find minimum-cost paths from a source vertex?

Spanning Tree: Definition

Definition

A spanning tree of a connected, undirected graph is a subgraph that includes all vertices, is connected and contains no cycles.

For V vertices:

$$\boxed{|E_T| = V - 1}$$

- All vertices are included.
- Exactly one simple path connects every pair of vertices.
- Removing any tree edge disconnects the tree.

Minimum Spanning Tree

Definition

A minimum spanning tree is a spanning tree whose total edge weight is minimum among all spanning trees.

$$W(T) = \sum_{e \in T} w(e)$$

Example

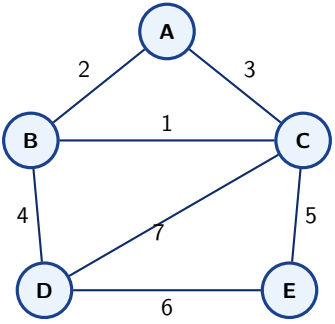
If selected edges have weights:

$$2,\ 4,\ 5,\ 7$$

then:

$$W(T) = 18$$

Weighted Graph Example



Question

Find a spanning tree with the minimum total weight.

Properties of Minimum Spanning Trees

1. An MST contains all vertices.
2. An MST contains exactly $V - 1$ edges.
3. An MST contains no cycles.
4. A connected weighted undirected graph has at least one MST.
5. Multiple MSTs may exist.
6. Distinct edge weights guarantee a unique MST.

MST: Cut Property

Cut Property

Consider any partition of the vertices into two nonempty sets.
A minimum-weight edge crossing the cut is safe to include in some MST.

Application

Prim's algorithm repeatedly selects a minimum-weight edge crossing from the current tree to a vertex outside the tree.

MST: Cycle Property

Cycle Property

If an edge is strictly heavier than every other edge in a cycle, that edge cannot belong to any MST.

Example

A cycle has edge weights:

$$2,\ 3,\ 8$$

The edge of weight 8 cannot belong to an MST.

Applications of MST

- Designing communication networks.
- Laying electrical cables.
- Connecting buildings with minimum wiring.
- Road and railway network planning.
- Connecting computer systems.
- Hierarchical clustering.

Goal

Connect all locations with minimum total connection cost.

Introduction to Prim's Algorithm

Definition

Prim's algorithm is a greedy algorithm that constructs an MST by starting from a vertex and repeatedly adding the minimum-weight edge connecting the current tree to an outside vertex.

- Starts from any vertex.
- Maintains one growing tree.
- Adds one new vertex per iteration.
- Avoids cycles automatically.

Why Is Prim's Algorithm Greedy?

At every iteration, Prim's algorithm selects:

Minimum-weight crossing edge

Greedy Choice

Select the cheapest edge connecting a tree vertex to a vertex not yet in the tree.

This local choice is safe because of the MST cut property.

Three Vertex Categories

During Prim's algorithm, vertices can be classified as:

1. **Tree vertices**
   Already included in the MST.
2. **Fringe vertices**
   Outside the MST but adjacent to at least one tree vertex.
3. **Unseen vertices**
   Not yet in the tree and not currently adjacent to any tree vertex.

Prim's Algorithm: Main Steps

1. Select any starting vertex.
2. Include it in the MST.
3. Examine edges connecting the tree to outside vertices.
4. Select the minimum-weight edge.
5. Add the new vertex and edge.
6. Repeat until all vertices are included.

Important

The selected edge must connect the current tree to a new vertex.

Prim's Algorithm: Pseudocode

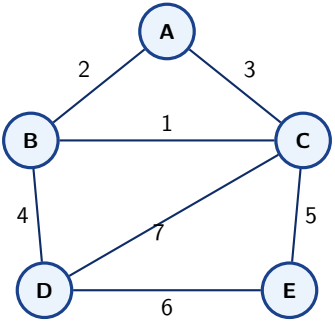
Algorithm: Prim(G, start)

```
For each vertex v:
  key[v] = INFINITY
  parent[v] = NIL
  inMST[v] = false
key[start] = 0
Repeat V times:
  u = vertex outside MST
  with minimum key[u]
  Add u to MST
  For each neighbor v of u:
    If v outside MST and
    weight(u,v) < key[v]:
      key[v] = weight(u,v)
      parent[v] = u
```

Important Arrays in Prim's Algorithm

Array	Purpose
`key[v]`	Minimum known edge weight connecting v to the current tree
`parent[v]`	Tree vertex through which v will be connected
`inMST[v]`	Indicates whether v has already joined the MST

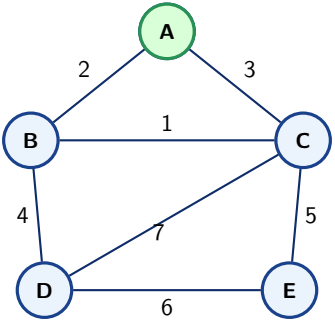
Worked Example: Prim's Algorithm



Task

Construct the MST using Prim's algorithm, starting from vertex A.

Step 1: Start at A



Initially:

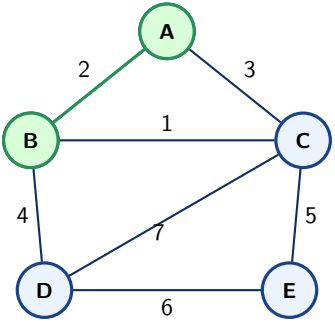
$$T = \{A\}$$

Candidate edges:

$$(A, B) = 2, \qquad (A, C) = 3$$

Choose:

Step 2: Add B



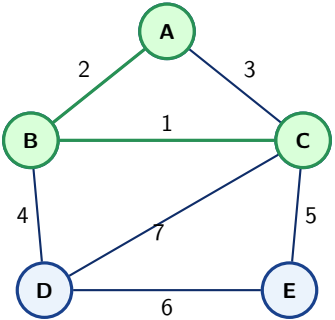
Current tree:

$$T = \{A, B\}$$

Candidate crossing edges:

$$(B, C) = 1$$

Step 3: Add C



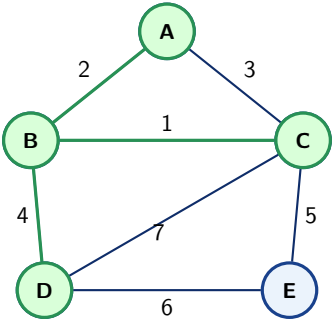
Current tree:

$$T = \{A, B, C\}$$

Candidate crossing edges:

$$(B, D) = 4$$

Step 4: Add D



Current tree:

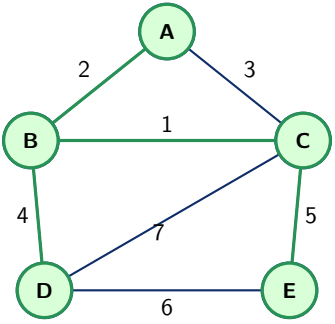
$$T = \{A, B, C, D\}$$

Candidate edges to E:

$$(C, E) = 5, \qquad (D, E) = 6$$

Choose:

Step 5: Add E



All five vertices are included.
Selected edges:

$$(A, B), (B, C), (B, D), (C, E)$$

$$\boxed{W = 2 + 1 + 4 + 5 = 12}$$

Prim's Algorithm: Complete Dry Run

Step	Added Vertex	Edge	Weight	Total
1	A	–	0	0
2	B	A–B	2	2
3	C	B–C	1	3
4	D	B–D	4	7
5	E	C–E	5	12

Minimum Spanning Tree Weight

$$\boxed{12}$$

Prim's Algorithm: Key Array Updates

After Selecting	A	B	C	D	E
Initial	0	∞	∞	∞	∞
A	0	2	3	∞	∞
B	0	2	1	4	∞
C	0	2	1	4	5
D	0	2	1	4	5
E	0	2	1	4	5

Note

Keys of vertices already selected are retained here for explanation; only outside vertices are eligible for the next selection.

C Implementation of Prim's Algorithm

We will use:

- Weighted adjacency matrix.
- Key array.
- Parent array.
- Boolean-style MST membership array.
- Minimum-key selection.

Matrix Convention

A weight of zero indicates no edge.
The example uses positive edge weights.

Prim in C: Part 1

```c
#include <stdio.h>
#include <limits.h>

#define V 5

int graph[V][V] = {
    {0, 2, 3, 0, 0},
    {2, 0, 1, 4, 0},
    {3, 1, 0, 7, 5},
    {0, 4, 7, 0, 6},
    {0, 0, 5, 6, 0}
};

int minKey(int key[], int inMST[]) {
    int min = INT_MAX;
    int index = -1;

    for (int v = 0; v < V; v++) {
        if (!inMST[v] && key[v] < min) {
            min = key[v];
            index = v;
        }
    }

    return index;
}
```

Prim in C: Part 2

```c
void primMST(void) {
    int parent[V];
    int key[V];
    int inMST[V] = {0};

    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        parent[i] = -1;
    }

    key[0] = 0;

    for (int count = 0; count < V; count++) {
        int u = minKey(key, inMST);

        if (u == -1) {
            printf("Graph is disconnected.\n");
            return;
        }

        inMST[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] > 0 &&
                !inMST[v] &&
                graph[u][v] < key[v]) {

                key[v] = graph[u][v];
                parent[v] = u;
            }
        }
    }
```

Prim in C: Part 3

```
    int total = 0;

    printf("MST Edges:\n");

    for (int i = 1; i < V; i++) {
        int u = parent[i];

        printf("%c - %c : %d\n",
               'A' + u,
               'A' + i,
               graph[u][i]);

        total += graph[u][i];
    }

    printf("Total MST Weight: %d\n", total);
}

int main(void) {
    primMST();
    return 0;
}
```

Prim's Algorithm: Expected Output

Program Output

```
MST Edges:
B - C : 1
A - B : 2
B - D : 4
C - E : 5
Total MST Weight:  12
```

Prim's Time Complexity

Adjacency Matrix Implementation

Finding the minimum key takes:

$$O(V)$$

Repeated for V vertices:

$$\boxed{O(V^2)}$$

Adjacency List with Binary Heap

$$\boxed{O((V + E) \log V)}$$

Prim's Space Complexity

For matrix-based Prim:

- Adjacency matrix: $O(V^2)$.
- Key array: $O(V)$.
- Parent array: $O(V)$.
- Membership array: $O(V)$.

$$\boxed{\text{Graph Storage} = O(V^2)}$$

$$\boxed{\text{Auxiliary Space} = O(V)}$$

Prim's vs Kruskal's

Feature	Prim's	Kruskal's
Strategy	Grow one tree	Merge components
Starting vertex	Required	Not required
Edge choice	Minimum crossing edge	Minimum safe edge globally
Cycle avoidance	Tree membership	Union-Find
Typical structure	Priority queue	Sorted edge list
Matrix time	$O(V^2)$	Not typical
Efficient implementation	Heap-based	Sorting + DSU

When Should We Use Prim or Kruskal?

Prim's Algorithm

Convenient when:

- Growing from a chosen starting vertex.
- Using an adjacency matrix.
- Working with dense graphs.

Kruskal's Algorithm

Convenient when:

- Edges are already listed.
- Working with sparse graphs.
- Union-Find is available.

Minimum Spanning Tree vs Shortest Path

MST

Minimizes the total weight needed to connect all vertices.

Shortest Path

Minimizes the total weight of a path between vertices.

Important

An MST does not necessarily contain the shortest path between a particular pair of vertices.

Shortest-Path Problem

Given a weighted graph:

$$G = (V, E)$$

with edge weights $w(u, v)$, find a path from source s to destination t minimizing:

$$\boxed{\sum_{e \in P} w(e)}$$

Examples

- Minimum travel distance.
- Minimum transportation cost.
- Minimum network routing cost.

Types of Shortest-Path Problems

1. **Single-source shortest paths**
   Find shortest paths from one source to every reachable vertex.
2. **Single-pair shortest path**
   Find a shortest path between two specified vertices.
3. **All-pairs shortest paths**
   Find shortest paths between every pair of vertices.

Shortest-Path Algorithms

Algorithm	Main Use
BFS	Unweighted graphs
Dijkstra	Nonnegative edge weights
Bellman-Ford	Negative edges allowed
Floyd-Warshall	All-pairs shortest paths

Today's Focus

Dijkstra's algorithm for single-source shortest paths.

Introduction to Dijkstra's Algorithm

Definition

Dijkstra's algorithm computes the minimum path cost from a source to every reachable vertex in a graph with nonnegative edge weights.

- Uses a greedy strategy.
- Maintains tentative distances.
- Repeatedly selects the vertex with the smallest tentative distance.
- Relaxes outgoing edges.

Important Condition

Dijkstra's algorithm requires:

$$\boxed{w(u, v) \geq 0}$$

for all edges.

Why?

With negative edge weights, a vertex considered finalized may later have a cheaper path through another vertex.

Bellman-Ford can handle negative edge weights when no reachable negative cycle makes shortest distances undefined.

Dijkstra's Main Idea

Maintain:

$$dist[v]$$

which represents the best known distance from the source to v.
Initially:

$$dist[source] = 0$$

For all other vertices:

$$dist[v] = \infty$$

Repeatedly select the unvisited vertex with minimum distance.

What Is Edge Relaxation?

Suppose an edge exists:

$$u \to v$$

with weight:

$$w(u, v)$$

If:

$$dist[u] + w(u, v) < dist[v]$$

then update:

$$\boxed{dist[v] = dist[u] + w(u, v)}$$

and:

$$parent[v] = u$$

Relaxation Example

Suppose:

$$dist[A] = 4$$

$$w(A, B) = 3$$

and:

$$dist[B] = 10$$

A path through A gives:

$$4 + 3 = 7$$

Since:

$$7 < 10$$

update:

$$\boxed{dist[B] = 7}$$

Dijkstra's Algorithm: Steps

1. Initialize source distance to zero.
2. Set all other distances to infinity.
3. Mark all vertices unvisited.
4. Select the unvisited vertex with minimum tentative distance.
5. Finalize that vertex.
6. Relax edges to its unvisited neighbors.
7. Repeat until all reachable vertices are finalized.

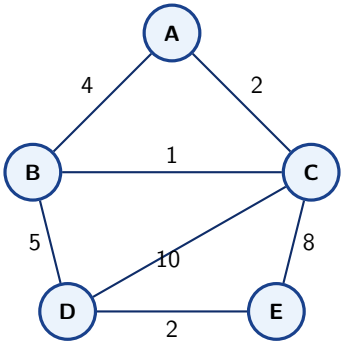
Dijkstra's Pseudocode

Algorithm: Dijkstra(G, source)

```
For each vertex v:
  dist[v] = INFINITY
  visited[v] = false
  parent[v] = NIL
dist[source] = 0
Repeat:
  u = unvisited vertex
  with minimum dist[u]
  If no reachable u exists:  Stop
  visited[u] = true
  For each edge (u,v,w):
    If v not visited and
    dist[u] + w < dist[v]:
      dist[v] = dist[u] + w
      parent[v] = u
```

Weighted Graph for Dijkstra

Consider this undirected weighted graph:



Task

Find shortest distances from A.

Step 1: Initialize Distances

Source:

$$A$$

Initial distances:

Vertex	A	B	C	D	E
Distance	0	∞	∞	∞	∞

Select A.
Relax its edges:

$$dist[B] = 4$$

$$dist[C] = 2$$

Step 2: Select C

Current distances:

$$A = 0, \quad B = 4, \quad C = 2$$

Select C because:

$$dist[C] = 2$$

Relax C–B:

$$2 + 1 = 3 < 4$$

Update:

$$\boxed{dist[B] = 3}$$

Relax C–D:

$$dist[D] = 2 + 10 = 12$$

Relax C–E:

Step 3: Select B

Current distances:

$$A = 0,\ B = 3,\ C = 2,\ D = 12,\ E = 10$$

Select B.
Relax B–D:

$$3 + 5 = 8$$

Since:

$$8 < 12$$

update:

$$\boxed{dist[D] = 8}$$

Step 4: Select D

Current distances:

$$A = 0,\ B = 3,\ C = 2,\ D = 8,\ E = 10$$

Select D.
Relax D–E:

$$8 + 2 = 10$$

Current:

$$dist[E] = 10$$

Since the candidate is equal, no update is required.

$$\boxed{dist[E] = 10}$$

Step 5: Select E

Select E.
No distance improves.
All vertices are finalized.

Final Shortest Distances

$$\boxed{A = 0, \quad B = 3, \quad C = 2, \quad D = 8, \quad E = 10}$$

Dijkstra: Complete Dry Run

Selected Vertex	A	B	C	D	E
Initial	0	∞	∞	∞	∞
A	0	4	2	∞	∞
C	0	3	2	12	10
B	0	3	2	8	10
D	0	3	2	8	10
E	0	3	2	8	10

Selection Order

$$\boxed{A, C, B, D, E}$$

Shortest-Path Tree

The parent array is:

$$parent[B] = C$$

$$parent[C] = A$$

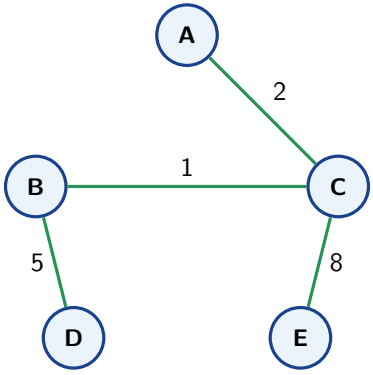
$$parent[D] = B$$

$$parent[E] = C$$

Shortest-Path Tree Edges

$$(A, C), (C, B), (B, D), (C, E)$$

Shortest-Path Tree: Diagram



Important

This is a shortest-path tree rooted at A, not necessarily an MST.

C Implementation of Dijkstra

We will use:

- Weighted adjacency matrix.
- Distance array.
- Visited array.
- Parent array.
- Minimum-distance selection.

Assumption

All edge weights are nonnegative.
Zero indicates no edge in this example.

Dijkstra in C: Part 1

```c
#include <stdio.h>
#include <limits.h>

#define V 5

int graph[V][V] = {
    {0, 4, 2, 0, 0},
    {4, 0, 1, 5, 0},
    {2, 1, 0, 10, 8},
    {0, 5, 10, 0, 2},
    {0, 0, 8, 2, 0}
};

int minDistance(int dist[], int visited[]) {
    int min = INT_MAX;
    int index = -1;

    for (int i = 0; i < V; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            index = i;
        }
    }

    return index;
}
```

Dijkstra in C: Part 2

```c
void dijkstra(int source) {
    int dist[V];
    int visited[V] = {0};
    int parent[V];

    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        parent[i] = -1;
    }

    dist[source] = 0;

    for (int count = 0; count < V; count++) {
        int u = minDistance(dist, visited);

        if (u == -1)
            break;

        visited[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] > 0 &&
                !visited[v] &&
                dist[u] <= INT_MAX - graph[u][v] &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
                parent[v] = u;
            }
        }
    }
```

Dijkstra in C: Part 3

```
    printf("Vertex Distance Parent\n");

    for (int i = 0; i < V; i++) {
        if (dist[i] == INT_MAX) {
            printf("%c INF -\n",
                   'A' + i);
        }
        else if (parent[i] == -1) {
            printf("%c %d -\n",
                   'A' + i, dist[i]);
        }
        else {
            printf("%c %d %c\n",
                   'A' + i, dist[i],
                   'A' + parent[i]);
        }
    }
}

int main(void) {
    dijkstra(0);
    return 0;
}
```

Dijkstra's Algorithm: Output

Expected Output

```
Vertex Distance Parent
A 0 -
B 3 C
C 2 A
D 8 B
E 10 C
```

Reconstructing a Shortest Path

To reconstruct a path from A to D:
Follow the parent array backward.

$$D \to B \to C \to A$$

Reverse the sequence:

$$\boxed{A \to C \to B \to D}$$

Total weight:

$$2 + 1 + 5 = \boxed{8}$$

Dijkstra's Time Complexity

Adjacency Matrix

Each iteration scans vertices to find the minimum distance.

$$\boxed{O(V^2)}$$

Adjacency List with Binary Heap

$$\boxed{O((V + E) \log V)}$$

Dijkstra's Space Complexity

For the matrix implementation:

- Graph matrix: $O(V^2)$.
- Distance array: $O(V)$.
- Parent array: $O(V)$.
- Visited array: $O(V)$.

$$\boxed{\text{Auxiliary Space} = O(V)}$$

excluding graph storage.

Prim's vs Dijkstra's Algorithm

Feature	Prim's	Dijkstra's
Goal	Minimum spanning tree	Shortest paths
Graph	Undirected, weighted	Directed or undirected
Greedy key	Cheapest connecting edge	Cheapest source distance
Output	MST	Shortest-path tree
Negative edges	Supported	Not supported
Matrix complexity	$O(V^2)$	$O(V^2)$

Applications of Dijkstra's Algorithm

1. GPS route planning.
2. Road network navigation.
3. Network routing.
4. Transportation optimization.
5. Robotics path planning.
6. Logistics and delivery routing.
7. Cost-based graph exploration.

Limitations of Dijkstra's Algorithm

- Does not support negative edge weights in the general case.
- Requires a suitable graph representation.
- Matrix implementation is inefficient for very large sparse graphs.
- Computes minimum path costs, not minimum spanning trees.

Important

For graphs with negative edge weights, consider Bellman-Ford.

Exercise 1: MST

Question

A weighted triangle has:

$$w(A, B) = 3$$

$$w(B, C) = 5$$

$$w(A, C) = 7$$

Find its MST weight.

Solution

Select weights 3 and 5.

$$\boxed{MST = 8}$$

Exercise 2: Prim's Algorithm

Question

Prim's algorithm starts at A.
Candidate crossing edges have weights:

$$4,\ 7,\ 2,\ 6$$

Which edge is selected next?

Solution

The minimum-weight crossing edge.

$$\boxed{2}$$

Exercise 3: Relaxation

Question

Suppose:

$$dist[u] = 6$$

$$w(u, v) = 4$$

$$dist[v] = 15$$

What is the updated distance?

Solution

$$6 + 4 = 10 < 15$$

$$\boxed{dist[v] = 10}$$

Exercise 4: Shortest Path

Question

Two possible routes exist:

$$A \to B \to D$$

with weights 3 and 4,
and:

$$A \to C \to D$$

with weights 2 and 8.
Which route is shorter?

Solution

$$3 + 4 = 7$$

$$2 + 8 = 10$$

MCQ 1

Prim's algorithm is used to find:

1. Shortest paths from one source
2. Minimum spanning tree
3. Topological ordering
4. Strongly connected components

Correct Answer

Option 2.

MCQ 2

Prim's algorithm selects:

1. The globally heaviest edge
2. The minimum-weight crossing edge
3. The vertex with maximum degree
4. The longest path

Correct Answer

Option 2.

MCQ 3

Dijkstra's algorithm requires:

1. All edges to be negative
2. Nonnegative edge weights
3. An unweighted graph only
4. A tree only

Correct Answer

Option 2.

MCQ 4

What does edge relaxation do?

1. Deletes an edge
2. Improves a tentative path cost
3. Sorts all vertices
4. Detects connected components

Correct Answer

Option 2.

MCQ 5

Matrix-based Dijkstra has time complexity:

1. $O(V)$
2. $O(\log V)$
3. $O(V^2)$
4. $O(E^2)$

Correct Answer

Option 3.

MCQ 6

Which statement is correct?

1. An MST always gives shortest paths.
2. Dijkstra constructs an MST.
3. Prim and Dijkstra optimize different objectives.
4. Prim requires negative weights.

Correct Answer

Option 3.

Complexity Summary

Algorithm / Implementation	Complexity
Prim, adjacency matrix	$O(V^2)$
Prim, binary heap	$O((V+E)\log V)$
Kruskal, sorting + DSU	$O(E \log E)$
Dijkstra, adjacency matrix	$O(V^2)$
Dijkstra, binary heap	$O((V+E)\log V)$

Key Takeaways

- An MST connects all vertices with minimum total edge weight.
- Prim grows one tree from a starting vertex.
- Prim selects the minimum crossing edge.
- Kruskal merges separate components.
- Dijkstra computes shortest paths from a source vertex.
- Dijkstra uses distance relaxation.
- Dijkstra requires nonnegative weights.
- MST and shortest-path objectives differ.

Questions for Revision

1. Define a minimum spanning tree.
2. Explain the cut property.
3. Write Prim's pseudocode.
4. Perform Prim's algorithm on a graph.
5. Compare Prim's and Kruskal's algorithms.
6. Define the shortest-path problem.
7. Explain edge relaxation.
8. Write Dijkstra's pseudocode.
9. Implement Dijkstra in C.
10. Compare Prim and Dijkstra.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 13: Graphs.
2. Thomas H. Cormen et al., *Introduction to Algorithms*.
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*.

Bellman-Ford, Warshall's and Floyd-Warshall Algorithms

- Negative-weight shortest paths.
- Bellman-Ford algorithm.
- Negative-cycle detection.
- Warshall's transitive closure.
- Floyd-Warshall all-pairs shortest paths.
- Detailed matrix dry runs.
- Complete C implementations.
- Algorithm comparisons and revision.

Student Feedback

Share Your Feedback



Thank you!